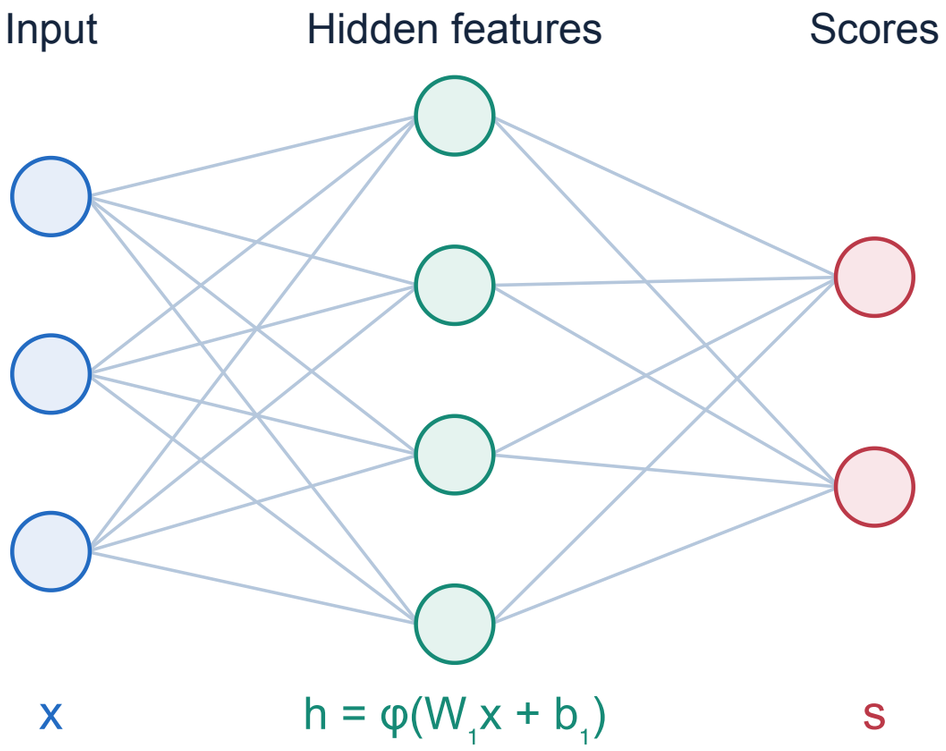
Deep Learning Basics

CS172 · Computer Vision I

Jiayuan Gu

ShanghaiTech University



From a learning objective to learned features

1. **Connect the objective to learning**

   Recall the loss-plus-regularization view from SVM.

2. **Optimize the parameters**

   Gradients, learning rate, and minibatch SGD.

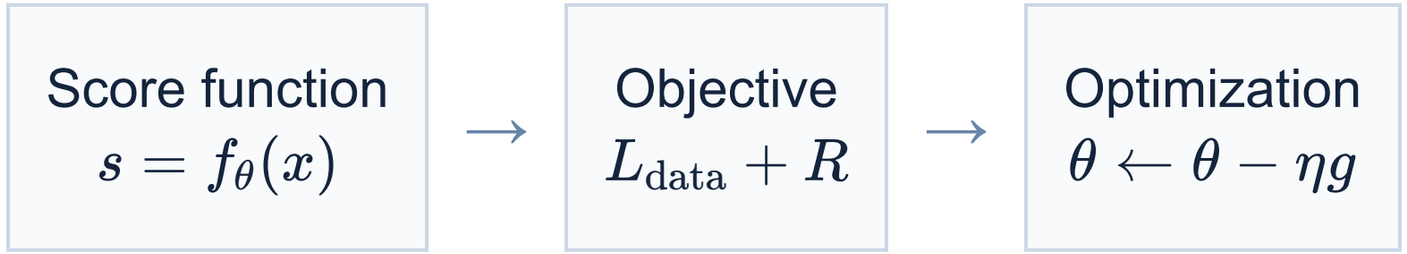
3. **Learn a nonlinear representation**

   MLPs, hidden layers, and activation functions.

4. **Compute gradients efficiently**

   Computational graphs, backpropagation, and the training loop.

Three pieces of a learning system



From SVM: a model, a loss, and a weight penalty define a learning problem.

> Today: understand SGD in detail, then learn the features as well as the classifier.

Fit the data while controlling the weights

$$L(\theta) = \underbrace{\frac{1}{N}\sum_{i} L_i}_{\text{data loss}} + \underbrace{\frac{\lambda}{2}\sum_{\ell}\|W_\ell\|_F^2}_{\text{L2 regularization}}$$

Data loss

Rewards scores that fit the training labels.

Regularization

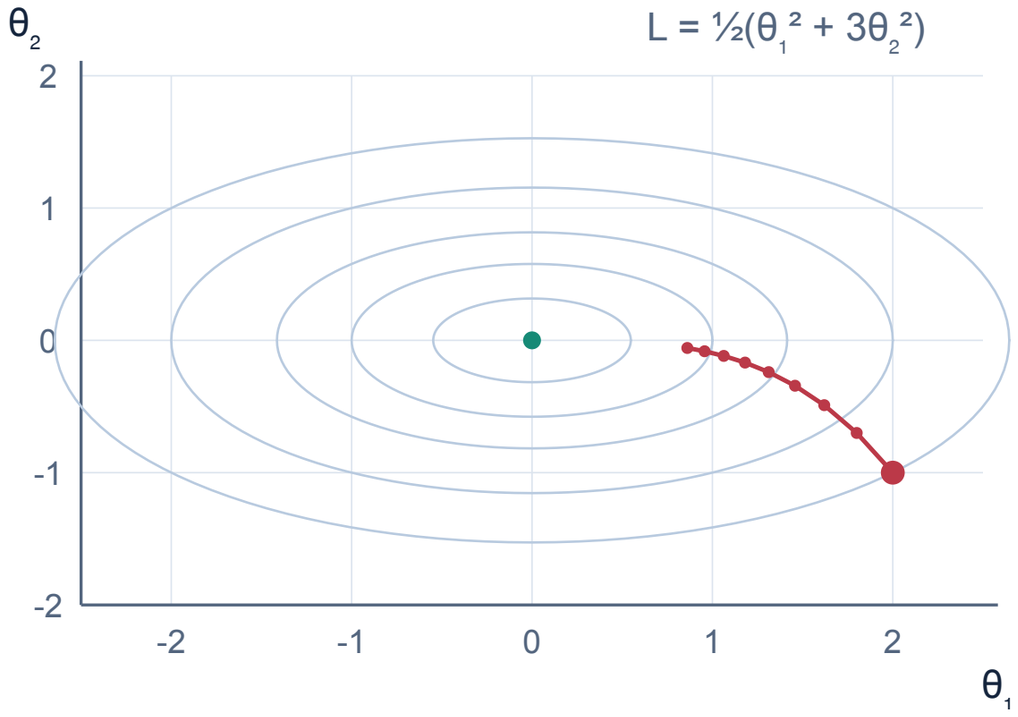
Discourages large weights; $\lambda \geq 0$ controls the tradeoff.

> Choose λ on validation data. Here biases are not regularized.

Optimization

We can measure a model’s error.
How do we change millions of parameters to reduce it?

Think of loss as a landscape



Each point is a parameter setting θ.

Height is the training objective $L(\theta)$.

> Use the local slope to choose a direction. Recompute it after every update.

A two-parameter illustration; real models can have millions of coordinates.

The negative gradient points downhill

$$\nabla_\theta L = \left[\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_P}\right]^\top$$

$$L(\theta + \delta) \approx L(\theta) + \nabla L(\theta)^\top \delta$$

$$\delta = -\eta \nabla L \quad \Longrightarrow \quad \Delta L \approx -\eta \|\nabla L\|_2^2$$

> For a sufficiently small positive step, move against the gradient.

Estimate a gradient numerically

Hold every other parameter fixed.

$$\frac{\partial L}{\partial \theta_k} \approx \frac{L(\theta + h e_k) - L(\theta)}{h}$$

e_k changes only coordinate k.

Setting	Loss
θ	1.25347
$\theta + 0.0001 e_1$	1.25322

$$\frac{1.25322 - 1.25347}{0.0001} = -2.5$$

Intuition and debugging: evaluating every coordinate is too costly for training.

Compute gradients; check with differences

Numerical differences

Approximate; many loss evaluations.

Useful for checking a few coordinates.

Analytic / automatic differentiation

Apply derivative rules to the computation.

Efficient enough for every training step.

$$g_k^{\text{check}} = \frac{L(\theta + he_k) - L(\theta - he_k)}{2h}$$

> Backpropagation will organize the chain rule for a neural network.

Gradient descent is a repeated update

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

$\theta_t = (2, -1)^\top$

$\nabla L = (2, -3)^\top$

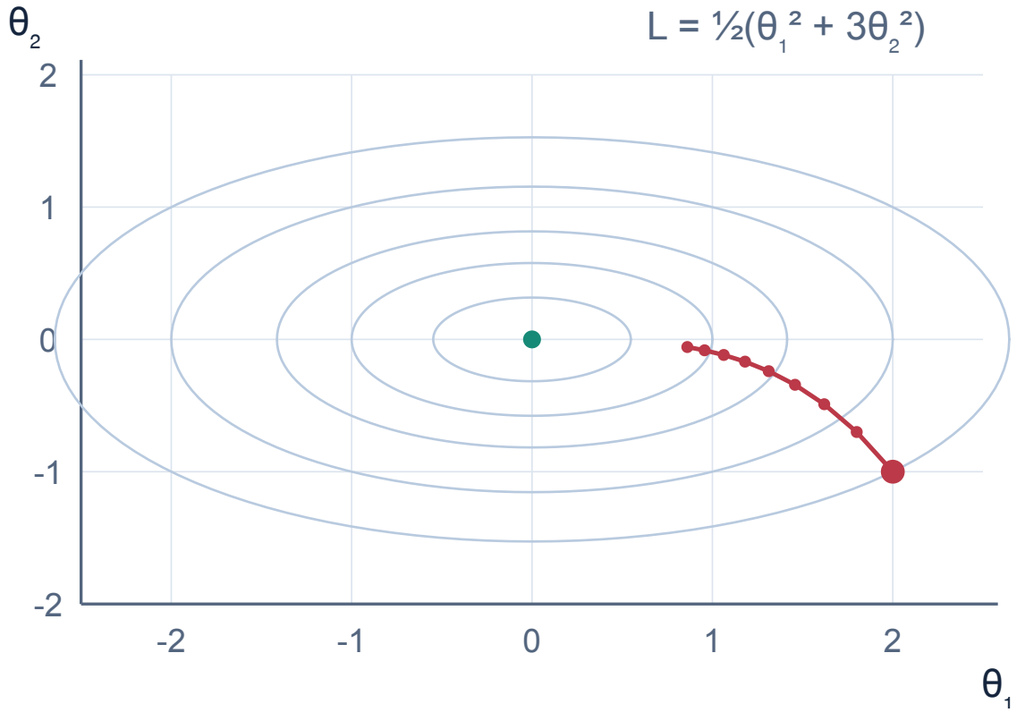
$\eta = 0.1$: learning rate

$$\theta_{t+1} = (1.8, -0.7)^\top$$

For $L = \frac{1}{2}(\theta_1^2 + 3\theta_2^2)$:

$3.5 \longrightarrow 2.355$

Learning rate controls the step



Learning rate η: **0.10**

$L = \frac{1}{2}(\theta_1^2 + 3\theta_2^2)$
Start: $(2, -1)$. Show 8 updates.

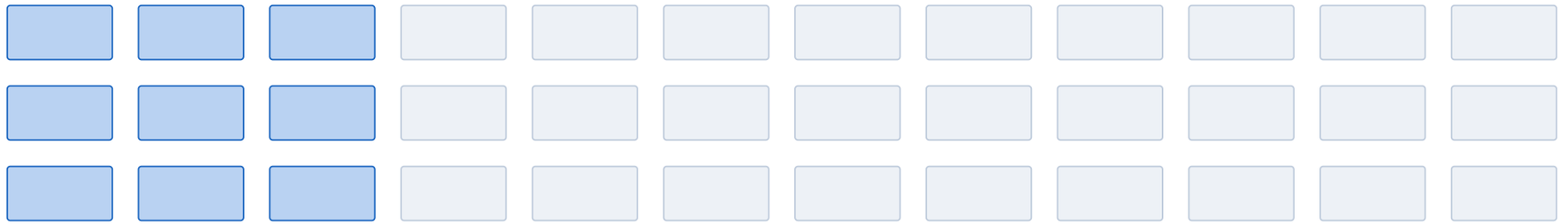
After 8 updates: L = 0.3756

Predict the path at $\eta = 0.6$, then 0.8.

The full gradient can be expensive

$$\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\theta) + \nabla R(\theta)$$

One full gradient needs contributions from all N training examples.



> Estimate the data gradient from a small random minibatch instead.

Stochastic gradient descent uses a minibatch

Sample a minibatch $\mathcal{B}_t$ of B examples.

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla L_i(\theta_t) + \lambda \nabla \left(\frac{1}{2} \sum_{\ell} \|W_\ell\|_F^2 \right)$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

> Uniform sampling gives an unbiased data-gradient estimate at fixed θ_t.

“SGD” commonly includes minibatches; strict single-example SGD has $B = 1$.

One minibatch, one concrete SGD step

Two sampled examples give data gradients:

$$g^{(1)} = (2, -4)^\top, \qquad g^{(2)} = (0, -2)^\top$$

Start at $\theta_t = (2, -1)^\top$, use $\eta = 0.1$, and set $\lambda = 0$.

$$g_t = \tfrac{1}{2}\left(g^{(1)} + g^{(2)}\right) = (1, -3)^\top$$

$$\theta_{t+1} = (2, -1)^\top - 0.1(1, -3)^\top = (1.9, -0.7)^\top$$

Illustrative sample gradients. This update need not reduce the full training loss.

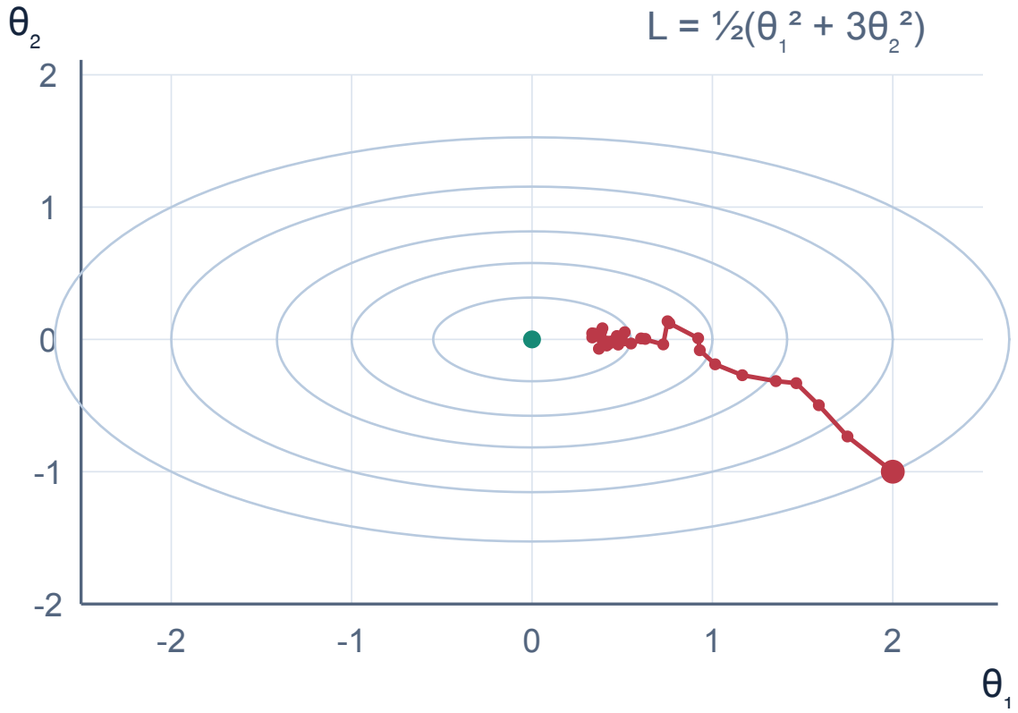
The SGD training loop

```
# X: (N, D), y: (N,); theta contains weights and biases
for epoch in range(num_epochs):
    order = rng.permutation(N)
    for ids in minibatches(order, batch_size):
        loss, grads = loss_and_grad(X[ids], y[ids], theta)
        for name in theta:
            theta[name] -= learning_rate * grads[name]
```

Iteration: one update. **Epoch:** one pass through the training set.

With $N = 50{,}000$ and $B = 100$, one epoch has 500 updates.

A minibatch trades computation for noise



Batch size B

8 examples

Same start, objective, and learning rate $\eta = 0.1$.

30 updates; fixed random sample sequence.

Synthetic quadratic: noise shrinks as the batch grows. Larger batches cost more per update.

Monitor learning, then validate choices

Loss rises or diverges

Check the learning rate, gradients, and numerical stability.

Loss barely changes

Check the learning rate, data pipeline, and gradient flow.

Training improves; validation worsens

Check overfitting, regularization, and stopping time.

> Use validation data to compare settings; reserve test data for final evaluation.

A noisy training curve is expected with minibatches. Inspect trends across many updates.

A brief look at other optimizers

Method	Main idea
Momentum	Accumulate past gradients to smooth the direction.
AdaGrad / RMSProp	Scale coordinates using past squared gradients.
Adam	Combine momentum and adaptive coordinate scaling.

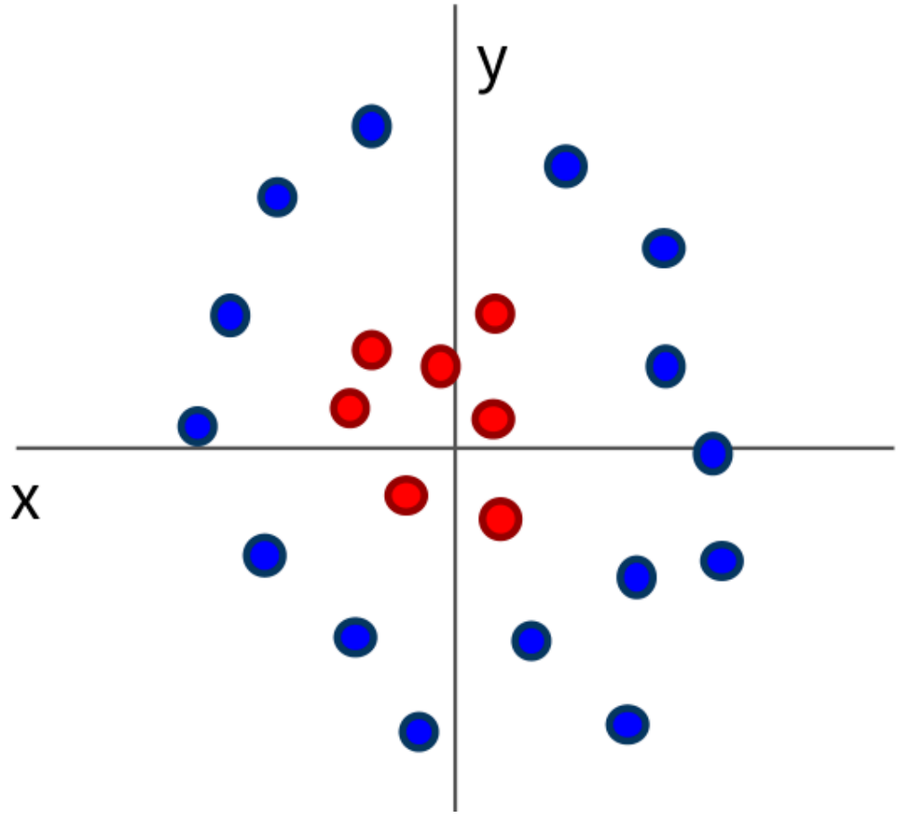
They address issues such as narrow valleys, uneven scales, and gradient noise.

> They still need a learning rate. No optimizer is best for every problem.

Neural networks

Keep the loss and the training loop.
Replace fixed features with a learned nonlinear mapping.

Linear boundaries have limits

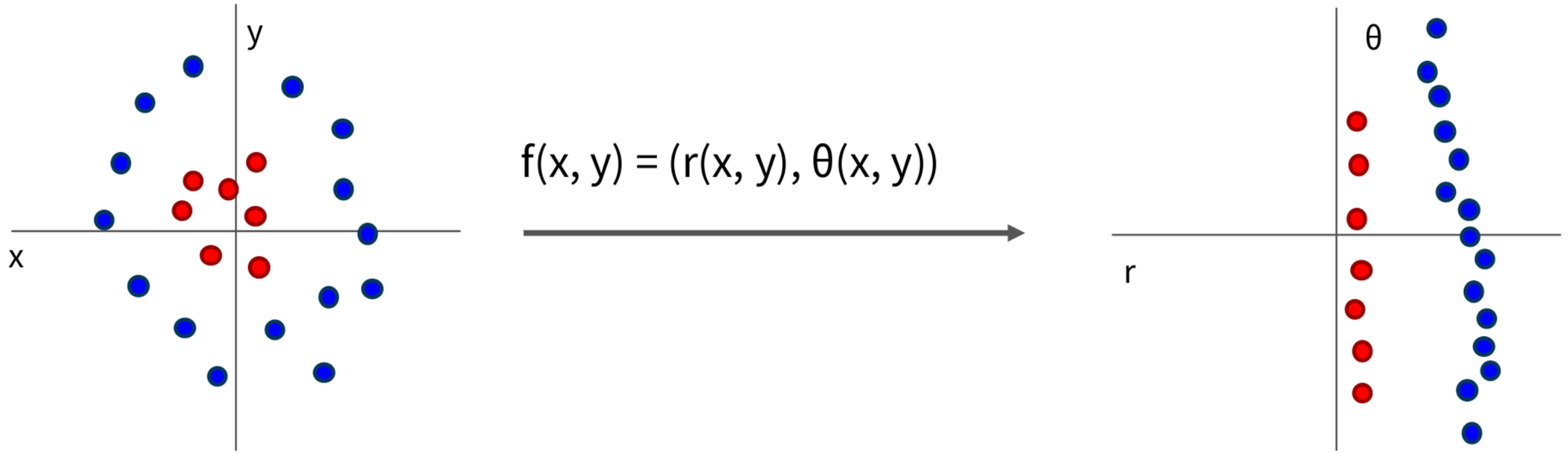


A linear score difference gives a hyperplane in the input coordinates.

Some classes require a nonlinear feature transform.

> Instead of choosing that transform by hand, can we learn it from data?

Transform features to separate classes

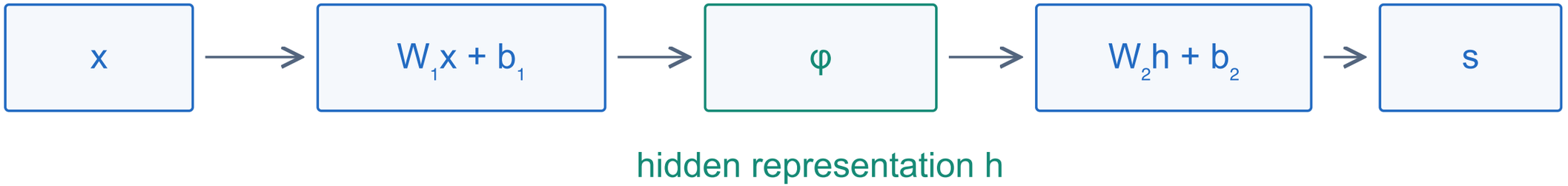


The classifier may be linear in $\phi(x)$ while nonlinear in the original input x.

Here the feature transform is designed by hand. An MLP learns its hidden features.

An MLP learns features, then class scores

$$h = \phi(W_1 x + b_1), \qquad s = W_2 h + b_2$$



MLP: multilayer perceptron; also called a fully connected network.

> The activation ϕ is nonlinear and acts element by element.

Affine layers alone still give an affine map

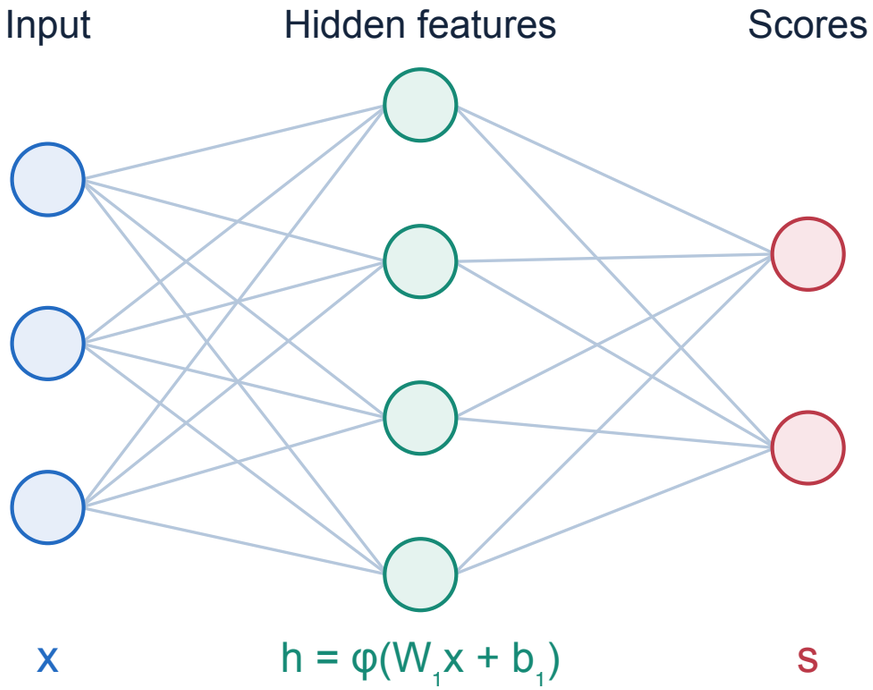
$$s = W_2(W_1 x + b_1) + b_2$$

$$s = \underbrace{W_2 W_1}_{W_{\text{eff}}} x + \underbrace{(W_2 b_1 + b_2)}_{b_{\text{eff}}}$$

> Without a nonlinear activation, extra layers do not create nonlinear decision boundaries.

With biases, “affine” is the precise term. The decision boundary is still a hyperplane.

Track the shapes through an MLP

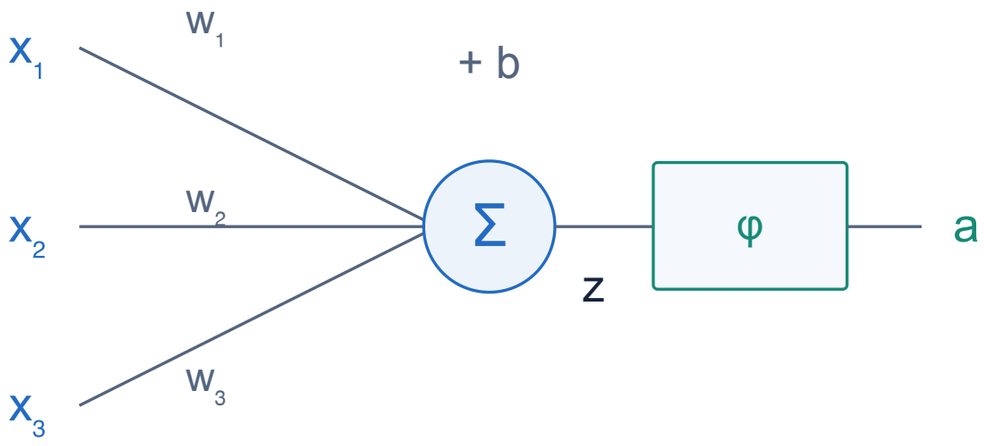


$x \in \mathbb{R}^D$, $h \in \mathbb{R}^H$, $s \in \mathbb{R}^K$.

Parameter	Shape
$W_1,\ b_1$	$H \times D,\ H$
$W_2,\ b_2$	$K \times H,\ K$

Parameter count:
$HD + H + KH + K$.

A neuron: weighted sum, then activation



$$z = w^\top x + b$$

$$a = \phi(z)$$

Weights select an input direction; the bias sets a threshold.

The biological analogy is loose. This is a mathematical computation.

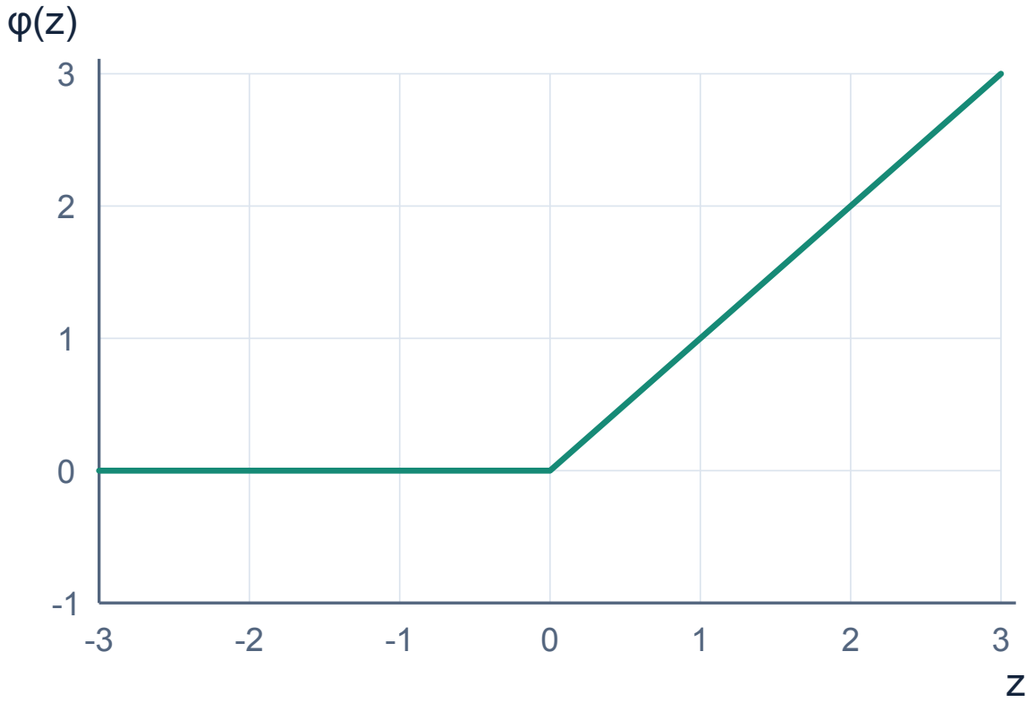
Calculate a complete forward pass

$$x = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_1 = 0 \qquad z = W_1 x = \begin{bmatrix} 3 \\ -3 \end{bmatrix}, \quad h = \mathrm{ReLU}(z) = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_2 = 0 \qquad s = W_2 h = \begin{bmatrix} 3 \\ -3 \end{bmatrix}$$

Predict class 1. If class 1 is true, softmax cross-entropy is $\log(1 + e^{-6}) \approx 0.00248$.

ReLU introduces a useful nonlinearity



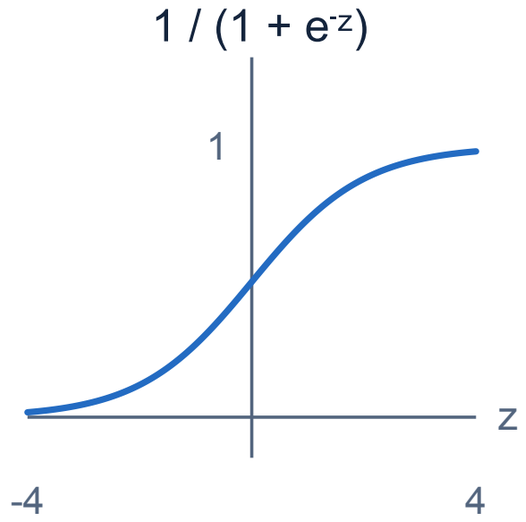
$$\mathrm{ReLU}(z) = \max(0, z)$$

Positive inputs pass through; negative inputs become zero.

Its slope is 1 on the positive side and 0 on the negative side.

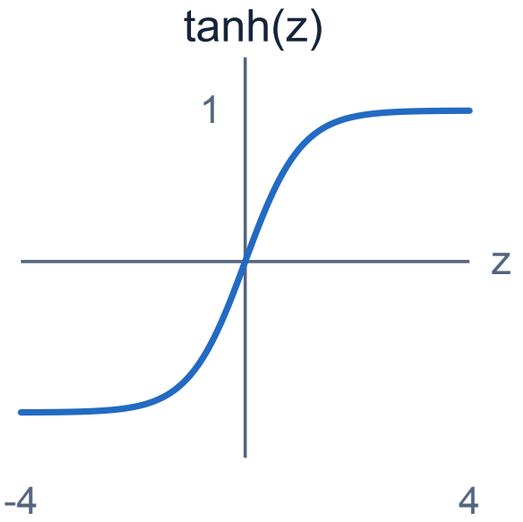
A unit that stays negative receives no gradient through ReLU.

Compare activation functions



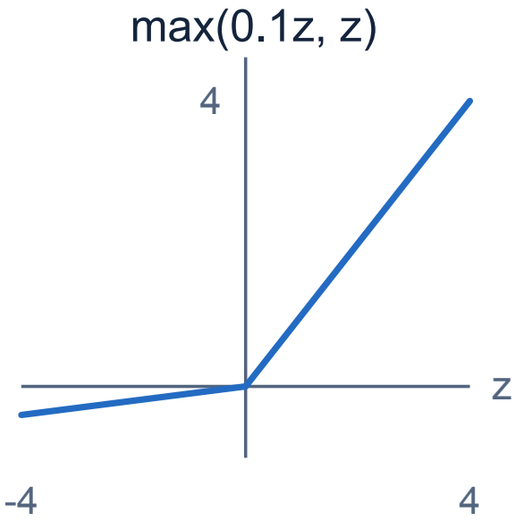
Sigmoid

Output in $(0, 1)$; saturates at both ends.



tanh

Output in $(-1, 1)$; also saturates.

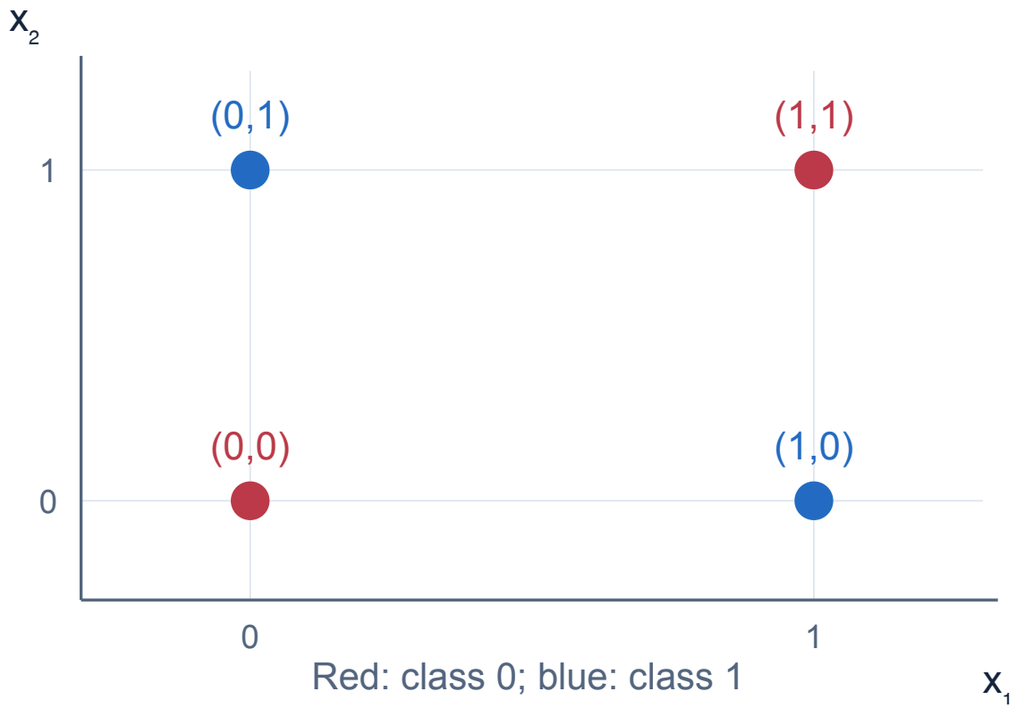


Leaky ReLU

A small nonzero slope for negative inputs.

These act inside hidden layers. Classifier output scores remain unrestricted.

Two ReLU features can solve XOR



Representation

Input coordinates (x_1, x_2)

$$h_1 = \max(0, x_1 - x_2)$$
$$h_2 = \max(0, x_2 - x_1)$$

Class 1 score: $s_1 = h_1 + h_2$.
Class 0 score: $s_0 = 0.5$.

The input points (0,0) and (1,1) coincide at hidden point (0,0).

Capacity and generalization

Width and depth enlarge the set of functions a network can represent.

Training fit

A flexible network can fit complicated patterns—and noise.

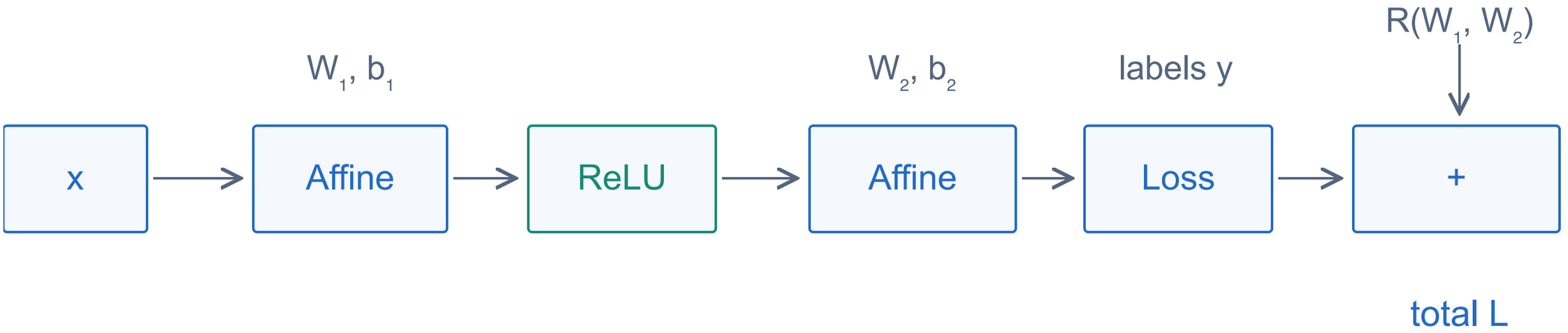
Validation performance

Use held-out data to choose capacity, regularization, and training time.

$$L(\theta) = \frac{1}{N}\sum_{i} L_i(f_\theta(x_i), y_i) + \frac{\lambda}{2}\sum_{\ell} \|W_\ell\|_F^2$$

> A lower training loss alone does not establish a better classifier.

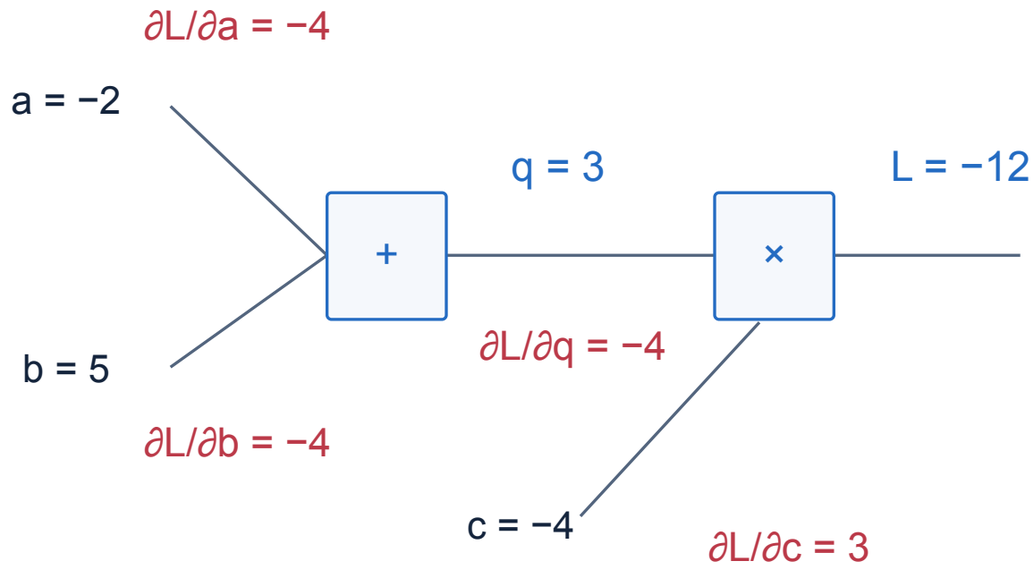
Represent computation as a graph



Nodes are operations. Edges carry intermediate values.

> Forward: compute the loss. Backward: compute how each parameter affects it.

Backpropagation follows the chain rule



A small scalar graph:

$$q = a + b, \qquad L = q\,c$$

$a = -2,\ b = 5,\ c = -4.$

Forward: $q = 3,\ L = -12.$

$$\frac{\partial L}{\partial a} = -4, \quad \frac{\partial L}{\partial b} = -4, \quad \frac{\partial L}{\partial c} = 3$$

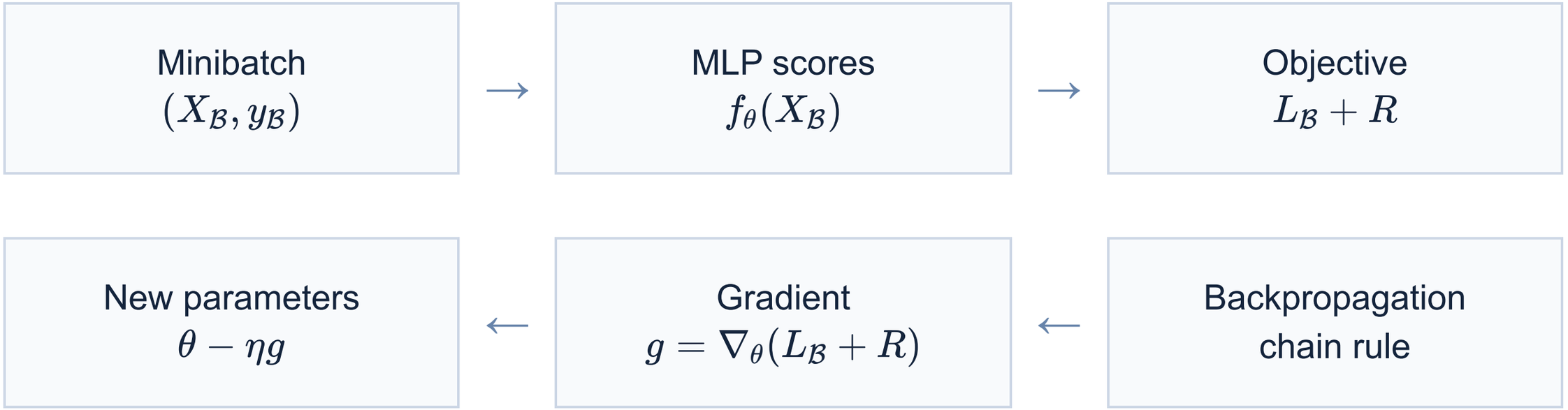
> At each node: upstream gradient × local derivative. Sum gradients from multiple paths.

Automatic differentiation in the training loop

```
# PyTorch sketch: scores are logits, labels are 0 ... K-1
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for xb, yb in loader:
    optimizer.zero_grad()
    scores = model(xb)
    loss = F.cross_entropy(scores, yb)
    penalty = sum((p * p).sum() for p in model.parameters()
                  if p.ndim > 1)  # this MLP's weight matrices
    (loss + 0.5 * lam * penalty).backward()
    optimizer.step()
```

Clear old gradients → forward → backward → update. Validate without parameter updates.

Connect the whole training system



> Architecture defines the computation; loss defines the goal; SGD makes the update.

Five ideas to take away

1. **Objective:** combine classification loss and weight regularization.
2. **SGD:** estimate a gradient from a minibatch, then update.
3. **MLP:** compose learned affine maps and hidden features.
4. **Nonlinearity:** activation functions prevent affine collapse.
5. **Backpropagation:** compute gradients by following the graph backward.

Check your understanding

1. An MLP has input dimension4 and hidden width3. What is the shape of W_1?
2. $\theta = (1, 2)$, minibatch gradient $g = (3, -1)$, $\eta = 0.1$: what is the next parameter vector?
3. What does removing every hidden activation do to an MLP?

1. 3×4. 2. $(0.7, 2.1)$. 3. An affine map.

Which operation computes the gradient, and which uses it?

Reading and references

- **CS231n 2026 · Lecture 3**

  Regularization → gradients → SGD. Other optimizers are supplementary.

- **CS231n 2026 · Lecture 4**

  Neural networks, nonlinear features, and computational graphs.

- **CS231n · Linear Classification · Optimization I · Optimization II**

  Softmax cross-entropy, gradient descent, and backpropagation.

- **Foundations of Computer Vision · Chapter 9**

  §9.5: empirical risk; §9.7.3: classification and softmax regression.